Malicious executables classification based on behavioral factor analysis

Hengli Zhao, Ming Xu, Ning Zheng
Institute of Computer Application Technology
HangZhou DianZi University
Hangzhou,China
zhenry86@163.com, mxu@hdu.edu.cn
nzheng@hdu.edu.cn

Jingjing Yao, Qiang Hou
The Third Research Institute of the Ministry of Public Security
ShangHai,China
e-mail: yjj@mail.trimps.ac.cn

***Abstract*—Malware is an increasingly important problem that threatens the security of computer systems. The new concept of cloud security require rapid and automated detection and classification of malicious software. In this paper,we propose a behavior-based automated classification method. Depends on behavioral analysis we characterize malware behavioral profile in a trace report. This report contains the status change caused by the executable and event which are transfered from corresponding *Win32 API* calls and their certain parameters. we extract behaviour unit strings as features which reflect diffierent malware families behavioral patterns. These features vector space servered as input to the *SVM*. We use string similarity and information gain to reduce the dimension of feature space. Comparative experiments with a real world data set of malicious executables shows that our proposed method can classify malware into diffierent malware families with higher accuracy and efficiency.**

Keywords- malware behaviors; malware classification; behaviour unit model;reducing dimensions;

I. INTRODUCTION

In the interconnected world of computers, malware has become an omnipresent and dangerous threat. Malware can infiltrate hosts using a variety of methods such as attacks against known software flaws, hidden functionality in regular programs, and social engineering. Given the devastating effect malware has on our cyber infrastructure, identifying malicious programs is an important goal. Detecting the presence of malicious code on a given host is a crucial component of any defense mechanism. Various obfuscation and polymorphism technology let the traditional static signature-based detection methods become inefficient and inadequate.The new concept of cloud security require rapid and automated detection and classification of malicious software. Malwares belong to the same family often shared the same behavior patterns because they often have the similar purpose and function. Malware is usually classified[1] according to its propagation method and goal into the following categories:

1. Viruses are programs that self-replicate within a host by attaching themselves to programs and/or documents that become carriers of the malicious code;
2. Worms self-replicate across a network;
3. Trojan horses masquerade as useful programs, but contain malicious code to attack the system or leak data;
4. Back doors open the system to external entities by subverting the local security policies to allow remote access and control over a network;
5. Spyware is useful software package that also transmits private user data to an external entity.

Combining two or more these malicious code categories can lead to powerful attack tools. For example, a worm can contain a payload that installs a back door to allow remote access. When the worm replicates to a new system (via email or other means), the back door is installed on that system, thus providing an attacker with a quick and easy way to gain access to a large set of hosts. Once the back-door tool gains a large installed base, the attacker can use the compromised host to launch a coordinated attack.Variants of malware family often had the same behavior because they often had the same purpose and function, so behavior-based method of malware classification is feasible.

In this paper, we develop a methodology for classfiying malware based on malware behaviour report. In summary, this paper makes contributions as follows:

- We characterize the behavioral profile through the behavioral traces of malware recorded in virtual environments. We generate the trace report, It contains the status change caused by the malware and event which are transfered from corresponding *Win32 API* calls and their certain parameters.
- We extract the various features by behaviour unit string from trace report and group many features into clusters based on the semantic correlation, then used these learned clusters as features and transform trace report into a high-dimensional vector space.
- We use support vector machine to learn from a training set and then we use the learned model for test set. Comparative experiment showes that our method has higher accuracy and efficiency.

The rest of this paper is organized as follows: Section 2 describes the related work. Section 3 describes malware behavioral analysis and trace report. Section 4 details the Suitable representation of trace report. Section 5 describes the experiment and evaluation. Section 6 concludes our work.

II. RELATED WORK

Related works on malware classification are focused on two aspects most close to our work. One aspect is finding accurate method of getting malware behavior report and the

978-0-7695-3948-5/10 $26.00 © 2010 IEEE
DOI 10.1109/IC4E.2010.78

TABLE 1. RELATIVE WINDOWS API USING OF WORM.WIN32.DOWNLOADER

Malicious intent	Calling data	Relative system call
Malware create and hide itself	File system	CreateFileA CopyFileA WriteFile SetFileAttributesA NtCreateFile …
Autorun after reboot	Windows registry	RegCreateKeyA RegOpenKeyExA RegQueryValueA RegSetValueExA …
Inject other process	Virtual memory\process	GetProcAddress WriteProcessMemory CreateRemoteThreat SetWindowsHookExA …
Downloader other malware connect remote server	Network connections	Listen recv send connect socket WSASocketA WSAConnect …
Protect malware itself	Other information	CreateMutexA CreateServiceA CheckRemoteDebuggerPresent …

other is generate suitable behavior representation of malware family as the input of machine learning techniques. Monitoring API call historial was used to discover program behavior[2], malicious program oftern run in a protective environment called 'sandbox' such as CWSandbox[3] and Anubis[4]. But sandbox's reporting features aren't perfect,it reports only the malicious program's visible behavior and not how it's programmed.

For the second aspect,[5][6]transform trace reports into sequences and use sequential distances to group them into clusters which are believed to correspond to malware families. The main shortage was due to the unsupervised learning of cluster, It can not use the guidance of the existing virus database. [7] presented a method closed to our method, they used text categories method to classify unknown malware samples based on their behavior report.But they ignored the semantic correlation of the feature word between variants of malwares which wil also lead to a high dimension vector space. Feature selection must be used to improve the classification efficiency.

III. MALWARE BEHAVIORAL ANALYSIS

Behavioral analysis focus on what programs do. The first step of our method is the automated analysis of malware samples. For this purpose, we have extended *Argus*[8], our tool for automated dynamic malware analysis. Because malware often pose strong threat to the computer system and the local network, so we divide the analyzer system in two operator system. One is host and the other is guest system. We use Vmware to realized this architecture. *Argus* is running within in the Vmware which provides a tightly controlled set of resources for programs to run in, such as network access, virtual space on disk and memory. The ability to inspect the host system or read from input device could be usually disallowed or heavily restricted[9]. The analysis system let malware execute in the emulated environment for limited time, commonly one or two minutes. Malware will invoke many system calls to do their malicious actions .For example, trying to modify certain parts of the system registry, or write to pre-defined folders. The action can be blocked, or the user notified about the attempted action. Combining *API* hooking and *DLL* injection within the vmware, our analyzer tool can trace and monitor all relevant system calls during the running of the malware. In Windows system these system calls often closely relative windows *API*.

For example, '*Worm.Win32.Downloader*' is a worm spreading via the Internet. It infected the host system and finishes installing snugly, but it must step by step as follows:

1. Create and Copy itself to the Windows directory under the name 'fvprotect.exe' and hide itself.
2. Autorun after reboot.
3. Inject and start system executeables like "C:\WINDOWS\system32\dwwin.exe".
4. Downloader other malcious executeables.
5. Protect itself. If the worm finds the keys listed below in the system registry key,It will delete them.

Table 1 is a relative windows *API* using of this malware samples when it executed in a computer.

These *API* calling data generates structural reports, and we could decide whether an unknown program is malicious based on modeling of these report [6]. Our report provides a high-level summary of the observed events and often more than one related *API* call is combined into a single operation.Of course parameters of these functions are also very important because these parameters will Directly reflect the behavior of executeables is benign or malicious, and often indicates a kind of malware family behavioral feature. For example, Figure 1 (a) is Very suspicious as a malware but (b) seem have a low risk.

CreateFileA()∧%system32%∧%PE file% (a)
CreateFileA()∧%Common directory%∧%.txt% (b)

Figure 1. conjunction of API names and parameters

we also meet the problem that a program's network activity is not sufficiently captured by trace the relative windows network *API*, so we built a network capture component through the *Wincap*, this tool can capture and transmit network packets bypassing the protocol stack, and has additional useful features. With its help we can recognize and parse application-level protocols. So we can learn detailed network information such as what malware has done based on the http protocol.

Through this way will will produced a readable and easy-understand trace report like Figure 2. The trace reports provide detailed information about malware behavior, but raw reports are not suitable for application of learning techniques. In next section we will using behaviour unit model to transform these report to a high-dimensional feature space.

file activities
CreatFile "C:\DOCUME~1\user\LOCALS~1\Temp\$$a1.bat"
CreatFile "C:\WINDOWS\Logo1_.exe"
process activities
CreatRemoteThread "C:\WINDOWS\system32\cmd.exe"
CreateProcess "C:\WINDOWS\Logo1_.exe"
WriteProcessMemory "C:\WINDOWS\Logo1_.exe"
registry activities
reg_modification "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Logo1_.exe"
reg_query_value "HKCU\Software\Microsoft\Command Processor" value="AutoRun"
Network
Udp_connection port 23413
system information
Createservice "net1.exe"
Createmutex "CTF.Asm.MutexDefaultS-1-5-21-1229272821-1004336348-527237240-1003"

Figure 2. Trace report example

IV. SUITABLE REPRESENTATION OF TRACE REPORT

A common problem encountered with learning from opaque data is finding a suitable representation to transfer opaque data into vectorial data. In our paper we use a behaviour unit model and bag of word model to finish this work.

a trace report record the behaviour of a program with corresponding situation of *API* calls. behavioral patterns such as opening a file, creating a mutex, or setting a registry key could be extracted from the trace reports, we express this patterns in the form of *OS* objects and the corresponding *API* call function that are invoked on these objects. In this way we can transform each behaviour into a pair of subject. We define behaviour unit B_1 as $B_1 = (O, OP)$ where O is the set of all *OS* objects, OP is the set of all security relative operations carried out on the *OS* objects. Of course many behaviour don't need to carry out on a *OS* object such as *ZwQuerySystemInformation*. So we also define $B_2 = (OP)$. An *OS* object represents an entity in computer, we divid these *OS* object into different type such as file, registry, process, service. especially file types, covering a variety of types, including name pipes, drivers mailslot resource. *OS* objects also contains many synchronization resource, such as operations on semaphores and mutexes. We also select security relative windows *API* functions and map them into *OS* operations. We ignore unimportant details such as the file read operation invoked by *ReadFile* or *ReadFileEx* and map these two system calls into the single *OS* operation *ReadFile*. But we don't map the function of *Winexec* and *Createprocess* into an single *Createprocess* operation because these infomation is very important for the malware classification. The windows platform now has more than 1,000 *API* functions. Currently we selected 189 security-related *API* functions which also used by malware frequently. Then we map these *API* functions to 68 *OS* operations. We also take into account other important parameters which is not a *OS* object in *API* functions, they also reflect the behavioral pattern of a malware family. so we define P where the P represent the important parameters in *API* functions.

Our approach builds on the vector space model and bag-of-words model. A trace report is characterized by frequencies of contained unit B_1, B_2 and P (these three unit will all be processed in the form of string). We refer to the set of considered unit as feature set F and denote the set of all possible reports by T . Given a string $s \in F$ and a report $t \in T$, we determine the number of occurrences of s in t and obtain the frequency $f(t,s)$ .The frequency of a string s acts as a measure of its importance in t . And the value of $f(t,s)$ will be taken as the value of corresponding feture s in the vector space of report t . We derive function φ which maps trace reports to an $|F|$-dimensional vector space by considering the frequencies of all strings in F :

$$\varphi : T -> \mathbb{R}^{|F|}, \varphi(t) \mapsto (f(t,s))_{s \in F}$$

For example, the *OS* object "C:\WINDOWS\Logo1_.exe" in figure 2 is carried out by three behavior unit. So there will be three dimensions in the resulting feature vector space and the feature value correspond to the frequencies of these behaviour unit strings in trace reports.

At a first glance, this method will generate a high-dimensional vectors as F may contain many arbitrary strings, especially feature strings which contain OS object cann't be known in advance. We can not define a feature set F a priori simply. But we can preprocess these strings and group into clusters based on their similarity. For example, malware of *trojan.psw.win32.mapdimp* family usually created *midimap??.dll* and *didimap??.dat* files in the system directory (?? represents random letters). If we adding each file into feature space, this will result in a large number of redundant features. So we adapt the *Levenshtein Distance Algorithm*[11] to measure the similary between these feature string which belong to same type. Combination knowledge

of security expertise and our test experiments, we chosen different distance threshold value for different *OS* object type. For file type We set the threshold with 3 and for register type, we calculate the ratio of distance value and corresponding length of registry entry and set the threshold value with 0.89. We will group similar strings into a string cluster when their similar less than corresponding threshold value . Then we will use the learned clusters as our features. Note that our preprocess is only process same type *OS* objects and parameters, do not deal with *OS* operations OP .

The resulting feature set F correspond to various strings of possible operations, *OS* object and with the frequency values we generated the vector data from malware report. Next we will apply learning algorithms on the vector data and classify the malware into diffierent malware families.

V. EXPERIMENT AND EVALUATION

We now proceed to evaluate the performance and effectiveness of our methodology. Our malware collection used for learning and subsequent classification of malware behavior consists of 13223 malicious executables. Malicious executables are provided by *ANTIY* laboratory[12], a member of *CNCERT/CC*.(National Computer Network Emergency Response Technical Team/Coordination Center of China). These malicious software covers current popular malware families such as Worm.Win32.Viking.

1:Worm.Win32.Downloader (313)	7:Backdoor.Win32.Agent(363)
2:Worm.Win32.Viking(659)	8:Virus.Win32.Parite(306)
3:Trojan-Dropper.Win32.Small(364)	9:Worm.Win32.AutoRun(334)
4:Email-Worm.Win32.Bagle(327)	10:Virus.Win32.Virut(366)
5:Trojan-Proxy.Win32.Ranky(321)	11:Worm.Win32.Otwycal(326)
6:Backdoor.Win32.Agobot(317)	

TABLE 2. MALWARE FAMILIES ANTIVIRUS SCANER BY VIRUSTOTAL

However, the field of malware classification research is made vastly more complicated by the fact that the same piece of malware may be given different names by its creator and by different antivirus companies. For example, a Trojan horse was defined as "*Trojan.Peacomm*" by Symantec laboratory, but the same malware is named " *Trojan.Small.DAM*" by F-Secure Lab,but the Trend Lab defined as "*TROJ_SMALL.EDW* ". So we use VirusTotal [14] scan all these malware samples, VirusTotal is a service that analyzes suspicious files which combined with 40 professional antivirus engines. We only select malware samples which at least have the same malware family information through scan of more than 5 antivirus engines. Through this pre-scanner, we get a data set of 3996 malware samples with 11 classes from total 13223. The ratio is about 30%. The details of data set as shown in Table 2. These samples are randomly split into two partitions, a training and testing partition, training contains 2664 samples and test is 1332. For each partition behavior-based trace reports are generated and transformed into a vectorial representation as discussed in Section 4. The training partition is used to learn individual *SVM* classifiers with different parameters for regularization and kernel functions. Finally, we used the learned svm model on the testing partition.

In our experiment we choose Radial Basic Function in our *SVM* model. *Konrad Rieck*[7] Used a similar approach to classificate the malware(expressed as *KR* method), they note that their method is generic and can be easily adapted to different report formats of malware analysis software. So we perform their method on our dataset. We can make a comparison with their method. They didn't give the details about the feature vector space, so we adopt the attributes reduction of *Information-Gain*[13] on the feature sets with these two method. We calculate the Information Gain of each feature:

$$IG(j) = -\sum_{v_j \in (0,1)} \sum_{c \in \{c_j\}} P(v_j, C) \log \frac{p(v_j, C)}{p(v_j)p(C)}$$

Where $IG(j)$ denote the Information Gain value of feature j , C represents one class in $\{C_i\}$, $\{C_i\}$ represent the class set of malware family. $p(v_j, C)$ denotes the probability of feature j with the value of v_j in class C . $p(v_j)$ denotes the probability of feature j equal v_j in all training sets. $p(C)$ denotes the probability of class C in all training sets. At last we will select features which have the lower IG value and save in the feature database.

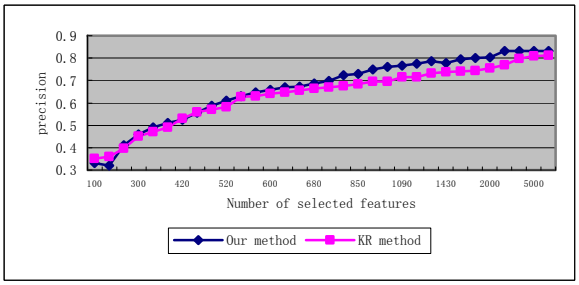

Figure 3. The result of comparison with two method

Because we could not decide the size of feature database, so we make experiment in different feature size level. We use the conventional Precision to measure the performance of method. Figure 3 shows the experimental results. In figure 3, the precision represents the average precision of total malware families.

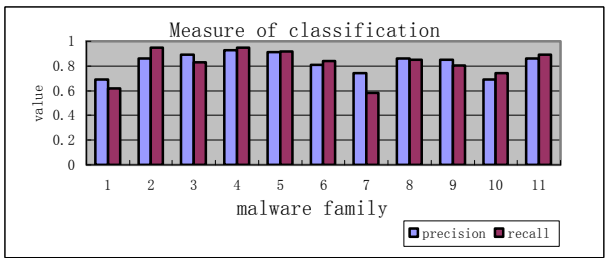

Figure 4. Performance of malware classification per family

From Figure 3 we can find that our method provides better performance than *KR* method in most different features size cases. With 960 features, Our method provides 5.8% higher than *KR* method. Only with twice size of selected features *KR* method can provide the similar performance to Ourmethod. Figure 4 provides the per-family precision and recall achieved on the 1332 test malware samples. On average, 83.3% of the malware is classified correctly. Some malware is classified with high accuracy such as *Email-Worm.Win32.Bagle*(4), while on the other hand malware families such as *Worm.Win32.Downloader*(7) and *Virus.Win32.Virut* (14) are perform poorly. Manually analysis some samples of these two malware families found that these two family malware contains more different and random behaviour details.

VI. CONCLUSION AND FUTURE WORK

In our paper we generate a trace report based on the *API* calling data and we also use a learning-based approach to automatic classification of malware. we use create a model with the principle of *VSM* to extract features which reflect diffierent malware behavioral patterns. Meanwhile,we compared our method with other similar approach. Experiment indicate that our method perform better. A limitation of dynamic malware analysis based on *API* trace report is not robust enough especially faced the malware which used many anti-tracing technology. Meanwhile the scalability of our technique is limited especially face These issues must be met in the future work.

REFERENCES

[1] G. Mc Graw, G. Morrisett, "Attacking malicious code: Report to the infosec research council", IEEE Software, 17(5) , Sept 2000, pp. 33-41.

[2] Wagner M. (2004). Behavior Oriented Detection of Malicious Code at Run-time. M.Sc. Thesis, Florida Institute of Technology.

[3] C.Willems, T.Holz, and F.Freiling. Toward Automated Dynamic Malware Analysis Using CWSandbox. IEEE Security and Privacy, 2007, 5(2):32-39.

[4] U.Bayer, C.Kruegel, and E.Kirda. TTanalyze: A Tool for Analyzing Malware. In 15th Annual Conference of the European Institute for Computer Antivirus Research, Hamburg, Germany, 2006: 180−192.

[5] M.Bailey,J.Oberheide,J.Andersen,Z.M.Mao,F.Jahanian,andJ.Nazario. Automated classification and analysis of internet malware. In Proceedings RAID07,pages 178–197,2007.

[6] Tony Lee & Jigar J. Mody Behavioral Classification .In Proceedings of EICAR2006,April 2006.

[7] T.Holz,C.Willems,K.Rieck,P.Duessel,andP.Laskov.Learning and Classification of Malware Behavior.In DIMVA08,June2008

[8] Yongtao Hu Unknown Malicious Executables Detection Based on Run-Time Behavior In Fuzzy Systems and Knowledge Discovery, 2008 pp. 391-395.

[9] Wikipedia, "Sandbox" [Online], Available: http://en.wikipedia.org/wiki/Sandbox

[10] V. I. Levenshtein, Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady 10 (1966):707–710.

[11] Antiy Labs, "About Antiy Labs" [Online], Available: http://www.antiy.net/about/index.htm

[12] J. Han, M. Kamber, Data Mining : Concepts and Techniques, Morgan Kaufmann, August 2000

[13] VirusTotal Service: www.virustotal.com/